Here is a quick, copy-pasteable cheatsheet for the most common tasks using Matplotlib (`plt`) and Seaborn (`sns`).

1. The Setup

Always start with your imports and setting a base theme.

Python
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Set seaborn default theme (makes base plt plots look better too)
sns.set_theme(style="whitegrid")
```

2. Matplotlib (`plt`) Core Plots

Best for quick, simple plots and base-level customization.

Python
```
# Line Plot (trends over time)
plt.plot(x, y, color='blue', linestyle='--', marker='o', label='Trend')

# Scatter Plot (relationship between two variables)
plt.scatter(x, y, color='red', alpha=0.5)

# Bar Plot (comparing categories)
plt.bar(categories, values, color='green')

# Histogram (distribution of a single variable)
plt.hist(data, bins=20, color='purple', edgecolor='black')
```

3. Seaborn (`sns`) Statistical Plots

Best for plotting directly from Pandas DataFrames (`df`) and adding automatic aggregations/groupings.

Relational Plots (Continuous vs Continuous)

Python
```
# Scatter Plot with color grouping (hue)
sns.scatterplot(data=df, x='col_x', y='col_y', hue='category_col', size='size_col')

# Line Plot with automatic confidence intervals
sns.lineplot(data=df, x='time_col', y='value_col', hue='category_col')
```

Categorical Plots (Categorical vs Continuous)

Python
```
# Bar Plot (automatically calculates the mean and error bars)
sns.barplot(data=df, x='category_col', y='value_col')

# Count Plot (like a histogram for categorical data)
sns.countplot(data=df, x='category_col')

# Box Plot (shows distribution, quartiles, and outliers)
sns.boxplot(data=df, x='category_col', y='value_col', hue='another_category')

# Violin Plot (combines boxplot and kernel density estimate)
sns.violinplot(data=df, x='category_col', y='value_col')
```

Distribution & Matrix Plots

Python

```
# Histogram + KDE (Kernel Density Estimate) curve
sns.histplot(data=df, x='value_col', kde=True, hue='category_col')

# Heatmap (great for correlation matrices)
# Note: df.corr() generates the correlation matrix
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
```

4. Customizing the Plot

Customizing labels and titles is almost always done using Matplotlib, even if you used Seaborn to draw the plot.

Python

```
# Titles and Labels
plt.title("Main Plot Title", fontsize=16)
plt.xlabel("X-Axis Label", fontsize=12)
plt.ylabel("Y-Axis Label", fontsize=12)

# Axis Limits and Ticks
plt.xlim(0, 100)          # Set x-axis range
plt.ylim(0, 50)           # Set y-axis range
plt.xticks(rotation=45)    # Rotate x-axis text (good for dates)

# Legend and Grid
plt.legend(title="Legend Title", loc='upper right')
plt.grid(True, axis='y', linestyle='--') # Add horizontal gridlines
```

5. Displaying & Saving

Python

```
# Show the plot (required in standard scripts, optional in Jupyter)
plt.show()

# Save the plot to a file (MUST be called before plt.show())
plt.savefig('my_plot.png', dpi=300, bbox_inches='tight')
# bbox_inches='tight' prevents labels from getting cut off
```

6. Subplots (Multiple Plots in One Figure)

The modern way to arrange multiple plots using the Object-Oriented API (fig, ax).

Python

```
# Create a 1x2 grid of plots
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

# Plot on the first axis (axes[0])
sns.scatterplot(data=df, x='A', y='B', ax=axes[0])
axes[0].set_title("First Plot")

# Plot on the second axis (axes[1])
sns.barplot(data=df, x='C', y='D', ax=axes[1])
axes[1].set_title("Second Plot")

# Adjust spacing so they don't overlap
plt.tight_layout()
```

```
plt.show()
```